

Testing Y QA

Parte 9: “El costo de automatizar el testing”

Integrantes:

Romina Alejandra Diaz
Teodoro Welyczko
Francisco Daurat
Mateo Colombo
Ignacio Lorente Grignola

Profesor: Gabriel Hugo Taboada

1. Introducción.....	3
2. Elementos a evaluar.....	3
3. Estrategia de pruebas.....	3
4. Criterios de entrada y salida.....	4
5. Planificación y cronograma.....	4
6. Recursos necesarios.....	5
7. Riesgos asociados.....	5
8. Aprobaciones y responsables.....	6
9. El costo de automatizar el testing.....	7
Arquitectura y herramientas por etapa:.....	7
Etapas DevSecOps.....	7
Herramienta Propuesta.....	7
Justificación en el proyecto.....	7
CI / CD.....	7
GitHub Enterprise + Actions.....	7
Testing Automatizado (Funcional/E2E).....	7
Cypress.....	7
Seguridad (Sec).....	7
Fortify (SAST) + OWASP ZAP.....	7
Performance y Monitoreo.....	7
LoadRunner + Datadog.....	7
Estrategia de automatización:.....	8
Fuentes.....	8

1. Introducción

Objetivo: definir, en el marco del estándar IEEE 729, la planificación de pruebas para el sistema e-commerce de venta de libros de EMPRESA, integrando los artefactos elaborados en las Partes 1 a 6 del trabajo práctico y los principios del testing ágil del Módulo 9. El plan establece qué se prueba, cómo se prueba, quién lo prueba, cuándo y bajo qué criterios se acepta o se detiene la actividad.

Alcance: el plan cubre las pruebas funcionales y no funcionales sobre los módulos de Login, Catálogo/Búsqueda, Carrito, Pagos e Inventario, en los ambientes de Desarrollo, Testing/QA, UAT y Producción definidos en el Plan CM (Parte 4). Se incluyen pruebas unitarias, de integración, de sistema, de aceptación, de regresión y no funcionales (performance, seguridad, usabilidad, confiabilidad).

Contexto: EMPRESA es una organización mediana con aproximadamente 10 desarrolladores, sin proceso formal de aseguramiento de calidad previo. El Plan de Calidad (Parte 1), la checklist con criticidad Alta/Media/Baja (Parte 2), las herramientas de gestión (Parte 3), el Plan CM bajo IEEE 828 (Parte 4), los casos de prueba para login, carrito y confirmación de compra (Parte 5) y la estrategia de testing (Parte 6) constituyen la línea base sobre la que se construye este plan. En línea con los principios del Módulo 9, el proyecto adopta un enfoque de testing ágil, promoviendo la integración del tester al equipo, la automatización de pruebas de regresión y aceptación, iteraciones cortas y la capacidad de adaptación continua frente a cambios en los requerimientos.

2. Elementos a evaluar

Módulos, funciones y componentes bajo prueba derivados del relevamiento y del Plan CM:

Módulo	Funcionalidades bajo prueba	Criticidad
Login	Autenticación, validación de credenciales, control de acceso a la compra (Historia 1).	Alta
Catálogo / Búsqueda	Búsqueda por nombre, intérprete, precio y cantidad; visualización de ejemplares vendidos.	Media
Carrito	Agregar/eliminar libros, control de stock, actualización de total (Historia 2).	Alta
Pagos	Datos de envío, selección de método, validación de tarjeta/CVV, confirmación (Historia 3).	Alta
Inventario	Disponibilidad de stock, sincronización con catálogo (interfaz crítica del Plan CM).	Alta
Interfaz con sistema externo de pagos	Comunicación con proveedor externo, control de versión de API (Plan CM Parte 4).	Alta

3. Estrategia de pruebas

Se retoma la Parte 6 y se reorganiza bajo los lineamientos de IEEE 729 y los principios ágiles del Módulo 9. La

estrategia combina niveles tradicionales de prueba con prácticas ágiles (TDD, BDD, ATDD y testing exploratorio):

- **Pruebas unitarias:** ejecutadas por los desarrolladores aplicando TDD (escribir la prueba antes que el código). Verifican interfaces, estructuras de datos locales, condiciones límite, caminos independientes y caminos de manejo de errores en cada módulo.
- **Pruebas de integración:** estrategia incremental Bottom-Up en el orden Inventario + Catálogo → Carrito → Login → Pagos, con foco en las interfaces críticas definidas en el Plan CM.
- **Pruebas de sistema y BDD:** escenarios de punta a punta escritos en lenguaje natural (Dado-Cuando-Entonces) cubriendo el flujo Login → Carrito → Pago, validando comportamiento sobre implementación.
- **Pruebas de aceptación con ATDD:** el Representante de Negocio, los desarrolladores y el tester definen conjuntamente los criterios de aceptación antes de implementar cada historia de usuario; estos escenarios se automatizan.
- **Pruebas de regresión automatizadas:** factor clave del testing ágil; se ejecutan ante cada cambio para garantizar que lo nuevo no rompió lo viejo.
- **Pruebas exploratorias:** sesiones timeboxed sobre flujos nuevos o modificados, complementarias a las pruebas guiadas.
- **Pruebas no funcionales:** performance (respuesta < 3 s bajo carga normal), seguridad (rechazo de transacciones no autorizadas, protección de datos de tarjeta, inyección), usabilidad (checklist Parte 2) y confiabilidad (casos inválidos de Parte 5).

Técnicas de diseño de casos (Parte 5): clases de equivalencia (válidas/inválidas), análisis de valores límite (mínimo/medio/máximo) y resultado esperado. Herramientas: Jira (gestión de defectos y RFC), GitHub (versionado), Google Sheets (matriz de casos y checklist), pipelines de CI/CD para regresión automatizada y notificaciones por Slack/Teams.

4. Criterios de entrada y salida

Criterios de entrada (para comenzar)	Criterios de salida (para finalizar)
• Línea base de requisitos y casos de uso aprobada. • Ambiente de Testing/QA disponible y separado del de Desarrollo. • Release aprobado por el CM (Plan CM Parte 4). • Casos de prueba diseñados (Parte 5) y datos de prueba cargados. • Checklist de calidad (Parte 2) versionada. • Equipo de QA y desarrollo asignados.	• 100% de los casos de criticidad Alta ejecutados sin defectos pendientes. • Cobertura de regresión automatizada vigente. • Sin defectos abiertos de severidad Bloqueante o Crítica. • Métricas de performance cumplidas (< 3 s bajo carga normal). • Aprobación del Representante de Negocio en UAT. • Acta de aceptación firmada y release etiquetado en GitHub.

5. Planificación y cronograma

Se adopta el cronograma de 9 semanas con sprints de 2 semanas definido en el Plan CM (Parte 4), agregando los hitos específicos de testing. La estimación se basa en los requisitos de software, la opinión de expertos y proyectos anteriores (Módulo 8). Como referencia, la duración de testing se estima como un tercio de la

duración del desarrollo.

Actividad	Hito / Entregable	Semana
Diseño de casos y datos de prueba	Casos de Parte 5 versionados en repositorio	1
Pruebas unitarias y TDD continuo	Cobertura mínima por módulo en cada check-in	2 – 8
Pruebas de integración Sprint 1	Auditoría funcional Sprint 1	3
Pruebas de integración Sprint 2	Auditoría funcional Sprint 2	6
Pruebas de sistema y no funcionales	Release de testing aprobado por QA	7
UAT con Representante de Negocio	Acta de aceptación	8
Regresión final y release a producción	Entrega del sistema	9

6. Recursos necesarios

Recursos humanos: Responsable de QA (coordinador del plan), 1 tester ágil integrado al equipo, 4 desarrolladores (autores de pruebas unitarias), Líder Técnico, Project Manager, Representante de Negocio y CCB para cambios sobre la línea base. Bajo el principio de roles del Módulo 9, cada miembro asume habilidades de prueba.

Recursos técnicos: puestos de trabajo para el equipo de QA, servidor de almacenamiento de casos de prueba, servidor de versiones de fuentes, servidor de base de datos de prueba; ambientes separados de Desarrollo, Testing/QA, UAT (espejo de producción) y Producción.

Herramientas: GitHub (repositorio y versionado), Jira (gestión de RFC y defectos), Google Drive / Confluence (documentación), Google Sheets (matriz de casos y checklist), Slack / Teams (notificaciones), framework de pruebas unitarias en el lenguaje del proyecto, herramienta de automatización para regresión y aceptación, herramienta de pruebas de performance y de seguridad básica.

7. Riesgos asociados

Se identifican los principales riesgos del proyecto de testing siguiendo el formato del Módulo 7 (riesgo, efecto, acción preventiva, acción correctiva):

Riesgo	Efecto	Acción preventiva	Acción correctiva
Ambiente de testing no separado del de desarrollo.	Resultados inconsistentes y retrasos en la entrega.	Estructurar el ambiente de testing antes del comienzo de la etapa.	Estructurarlo lo antes posible.
Falta de tiempo para todos los tipos de prueba.	No detectar ciertos errores.	Incorporar al cronograma tiempos específicos por tipo de prueba.	Priorizar y ejecutar grupos de pruebas en paralelo.
Entrega no incremental por parte de desarrollo.	"Catarata de errores" y sobrecarga del equipo.	Considerar implementación incremental en el	Priorizar errores bloqueantes.

		cronograma.	
Cambios frecuentes de requerimientos (entorno ágil).	Retrabajo en casos de prueba.	Diseñar casos reusables y mantener regresión automatizada.	Replanificar prioridades en cada inicio de sprint.
Falla en la interfaz con el sistema de pagos externo.	Compromiso de la seguridad y la confiabilidad.	Registrar versión de API y aprobar cambios por CCB.	Ejecutar regresión sobre el módulo de pagos.
Inexistencia de herramienta de seguimiento de errores.	Pérdida de trazabilidad de defectos.	Usar Jira como base de datos de defectos.	Priorizar la administración de resultados de prueba.

8. Aprobaciones y responsables

Rol	Responsabilidad sobre el Plan de Testing	Aprobación
Responsable de QA	Elaborar y mantener el plan, coordinar al tester ágil, aprobar releases.	Firma
Tester ágil	Diseñar y ejecutar casos, participar en planificación y refinamiento.	Firma
Equipo de Desarrollo	Pruebas unitarias bajo TDD, corrección de defectos, soporte a integración.	Notificación
Líder Técnico (CM)	Aprobar línea base, generar releases, auditar repositorio.	Firma
Project Manager	Autorizar cambios de alcance y supervisar cumplimiento del plan.	Firma
Representante de Negocio	Definir criterios de aceptación, ejecutar UAT y aceptar el sistema.	Firma
CCB	Aprobar cambios sobre interfaces críticas (Inventario/Catálogo y Pagos).	Acta

9. El costo de automatizar el testing

Arquitectura y herramientas por etapa:

Etapas DevSecOps	Herramienta Propuesta	Justificación en el proyecto
CI / CD	GitHub Enterprise + Actions	Integración nativa con el versionado actual para orquestar pipelines ante cada check-in.
Testing Automatizado (Funcional/E2E)	Cypress	Ejecución rápida de la regresión automatizada y validación de los escenarios BDD/ATDD.
Seguridad (Sec)	Fortify (SAST) + OWASP ZAP	Análisis estático y dinámico automático para proteger la interfaz de pagos y datos de usuarios.
Performance y Monitoreo	LoadRunner + Datadog	Validación de métricas (< 3s bajo carga) y monitoreo de confiabilidad en UAT y Producción.

Justificación de la herramienta de testing (Enfoque AHP):

Para seleccionar Cypress como motor central de automatización, se aplicó un análisis multicriterio (AHP) comparándolo contra Selenium y Tricentis Tosca. Se evaluaron: Integración nativa CI/CD (peso 35%), Soporte para ATDD/BDD (30%), Capacidad de mantenimiento (25%) y Costo (10%). Cypress resultó la opción ganadora por su robustez técnica y acople perfecto con *pipelines* modernos.

Estrategia de automatización:

- Se automatizará el 100% de los casos de prueba de criticidad Alta (Login, Pagos, Inventario) y las pruebas *Sanity*.
- El *pipeline* de CI/CD ejecutará automáticamente pruebas unitarias y análisis de seguridad ante cada nuevo código introducido por los desarrolladores.
- Una vez superadas estas validaciones, el código pasará automáticamente al ambiente de Testing/QA para la regresión automatizada E2E con Cypress.

Fuentes

Universidad de Palermo, Facultad de Ingeniería (2024). Módulos 7, 8 y 9 de la materia Testing y QA: Estrategias de Testing, Plan de Prueba y Principios del testing ágil. Grupo 8 (2024-2026). Trabajo Práctico – Partes 1 a 6. Materia Testing y QA, Universidad de Palermo. IEEE Std 729 (Standard Glossary of Software Engineering Terminology) e IEEE Std 828 (Software Configuration Management Plans).